U08 · Ficheros y streams

Todo lo que has programado hasta ahora "olvida" sus datos en cuanto el programa termina. Esta unidad es sobre hacer que la información **sobreviva** al cierre del programa.

> **Dos cosas muy distintas que se llaman "Stream"**
>
> En la unidad 7 viste la **API Stream** (`java.util.stream.Stream`) — para procesar colecciones de forma declarativa. Aquí vas a ver los **streams de entrada/salida** (`java.io`, clases como `FileReader` o `FileWriter`) — un concepto mucho más antiguo, que representa un **flujo de datos** entrando o saliendo del programa (de un fichero, la red, el teclado…). Comparten nombre y poco más — no las confundas.

1. La clase `File`: representar una ruta

`File` no representa "un archivo" — representa una **ruta** del sistema de ficheros, exista o no en ese momento.

```
import java.io.File;

File carpeta = new File("C:/Temp");
File documento = new File("C:/Temp/notas.txt");
```

- **Ruta absoluta**: parte de la raíz del sistema (`C:/Temp/notas.txt`). **Ruta relativa**: parte de la carpeta desde la que se ejecuta el programa.
- Java acepta `/` como separador incluso en Windows, así que puedes usarlo siempre por simplicidad.

Comprobar, consultar y gestionar

```java
File f = new File("C:/Temp/notas.txt");

f.exists();          // ¿existe esa ruta?
f.isFile();           // ¿existe Y es un fichero?
f.isDirectory();       // ¿existe Y es una carpeta?
f.length();             // tamaño en bytes (solo tiene sentido en ficheros)
f.lastModified();        // fecha de última modificación, en milisegundos desde 1970

f.mkdir();               // crea la carpeta (falla si ya existe o el padre no
existe)
f.delete();                // borra el fichero, o la carpeta SI está vacía
f.renameTo(new File("nuevo.txt"));  // mueve y/o renombra

for (File hijo : carpeta.listFiles()) {   // listar el contenido de una carpeta
    System.out.println(hijo.getName());
}
```

2. Leer y escribir ficheros de texto

Leer: `FileReader` + `BufferedReader`

`FileReader` lee carácter a carácter; envolverlo en un `BufferedReader` te da `readLine()`, mucho más cómodo para trabajar con texto real:

```java
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Usa siempre **try-with-resources** (el `try (...)` con el recurso declarado entre paréntesis): cualquier clase que implemente `Closeable` se cierra automáticamente al salir del bloque, ocurra o no una excepción — sin necesidad de un `finally` con `close()` manual.

Escribir: `FileWriter` y `PrintWriter`

```java
try (FileWriter fw = new FileWriter("salida.txt")) {   // si existe, LO
SOBRESCRIBE por completo
    fw.write("Primera línea\n");
    fw.write("Segunda línea\n");
} catch (IOException e) {
    e.printStackTrace();
}
```

> ⚠ **Dos trampas clásicas de FileWriter**
>
> - **Sobrescritura silenciosa**: `new FileWriter("salida.txt")` borra el contenido anterior sin avisar. Si quieres añadir al final sin borrar, usa el segundo constructor: `new FileWriter("salida.txt", true)` (modo *append*).
> - **`write(int)` no es lo que parece**: `writer.write(65)` no escribe el texto `"65"` — escribe el **carácter** cuyo código es 65, es decir, la letra `A`. Para escribir un número como texto, conviértelo primero: `writer.write("" + 65)` o `writer.write(String.valueOf(65))`.

`PrintWriter` envuelve a `FileWriter` y añade métodos más cómodos, como `println(...)`, que además añade el salto de línea por ti:

```java
try (PrintWriter pw = new PrintWriter(new FileWriter("salida.txt"))) {
    for (int i = 0; i < 10; i++) {
        pw.println("Línea " + i);
    }
}
```

3. Serialización: guardar objetos completos

En vez de convertir tú mismo un objeto a texto y luego reconstruirlo, Java puede volcar el objeto **entero** a bytes (y reconstruirlo después) automáticamente, si la clase implementa `Serializable` (una interface “marcador”, sin métodos que implementar):

```java
class Persona implements Serializable {
    private String nombre;
    private int edad;
    // constructor, getters...
}
```

```java
// Escribir objetos
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("personas.dat"))) {
    oos.writeObject(new Persona("Juan", 49));
    oos.writeObject(new Persona("Susana", 45));
}

// Leer objetos
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("personas.dat"))) {
    Persona p = (Persona) ois.readObject();
    System.out.println(p);
} catch (EOFException e) {
    // fin del fichero: es la forma normal de saber que ya no quedan más objetos
}
```

4. La API NIO2: `Path` y `Files`

Desde Java 7, el paquete `java.nio.file` ofrece una alternativa más moderna a `File`: `Path` representa la ruta, y la clase `Files` concentra casi todas las operaciones como métodos estáticos — muchas de ellas en una sola línea:

```java
import java.nio.file.*;

Path ruta = Path.of("C:/Temp/notas.txt");

Files.exists(ruta);
List<String> lineas = Files.readAllLines(ruta);              // lee TODO el fichero
de golpe
Files.writeString(ruta, "Contenido nuevo");                    // escribe de golpe
Files.copy(ruta, Path.of("C:/Temp/copia.txt"));
Files.delete(ruta);

try (Stream<String> lineasStream = Files.lines(ruta)) {          // como un Stream de
la U07, línea a línea
    lineasStream.filter(l -> !l.isBlank()).forEach(System.out::println);
}
```

`Files.readAllLines`/`writeString` son cómodos para ficheros pequeños (cargan todo en memoria); para ficheros grandes, sigue siendo mejor procesar línea a línea con `BufferedReader` o `Files.lines(...)`.

5. Manipular XML

Java trae de serie (`javax.xml.parsers`) soporte para leer y escribir XML, principalmente con dos enfoques:

- **DOM (Document Object Model)**: carga el documento entero en memoria como un árbol de nodos que puedes recorrer y modificar libremente. Cómodo, pero consume memoria proporcional al tamaño del fichero.
- **SAX (Simple API for XML)**: lee el documento de un tirón, avisándote mediante eventos (“empieza esta etiqueta”, “termina esta etiqueta”…) a medida que lo procesa, sin cargarlo entero en memoria. Más eficiente para ficheros grandes, pero no puedes “volver atrás”.

```java
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(new File("datos.xml"));

NodeList alumnos = doc.getElementsByTagName("alumno");
for (int i = 0; i < alumnos.getLength(); i++) {
    Element alumno = (Element) alumnos.item(i);
    System.out.println(alumno.getAttribute("nombre"));
}
```

6. Manipular JSON

A diferencia de XML, el JDK estándar **no** incluye un procesador de JSON — hace falta una librería externa. Una de las más sencillas y populares es `org.json`:

```java
import org.json.JSONObject;

String texto = "{\"nombre\": \"Ada\", \"edad\": 28}";
JSONObject json = new JSONObject(texto);

String nombre = json.getString("nombre");
int edad = json.getInt("edad");

JSONObject nuevo = new JSONObject();
nuevo.put("producto", "Teclado");
nuevo.put("precio", 24.99);
System.out.println(nuevo.toString());   // {"producto":"Teclado","precio":24.99}
```

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA2. Utiliza estructuras de control y datos…	g) incorporado y utilizado librerías de objetos
RA3. Desarrolla programas estructurados…	g) comentado y documentado el código

RA	Criterios de evaluación cubiertos
RA4. Desarrolla programas utilizando objetos y clases…	j) conjuntos y librerías de clases
RA5. Realiza operaciones de entrada y salida de información…	a) consola para E/S · b) formatos de visualización · c) posibilidades de E/S del lenguaje · d) ficheros para almacenar y recuperar información · e) diversos métodos de acceso al contenido de los ficheros
RA6. Escribe programas que manipulen información con tipos avanzados de datos…	h) clases relacionadas con documentos XML · i) manipulaciones sobre documentos XML

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras